P4: 프로토콜 독립적 패킷 프로세서 프로그래밍

Pat Bosshart † , Dan Daly*, Glen Gibb † , Martin Izzard † , Nick McKeown ‡ , Jennifer Rexford, Cole Schlesinger**, Dan Talayco † , Amin Vahdat ¶ , George Varghese § , David Walker****

† Barefoot Networks, *Intel, ‡ Stanford University, **Princeton University, ¶ Google, § Microsoft Research

---

초록

P4는 프로토콜 독립적 패킷 프로세서를 프로그래밍하기 위한 고수준 언어이다. P4는 OpenFlow와 같은 SDN 제어 프로토콜과 함께 동작한다. 현재 형태의 OpenFlow는 동작하는 프로토콜 헤더를 명시적으로 지정한다. 이 헤더 필드 집합은 불과 몇 년 만에 12개에서 41개로 증가하여 명세의 복잡성을 높이면서도 새로운 헤더를 추가할 수 있는 유연성은 여전히 제공하지 못하고 있다. 본 논문에서는 OpenFlow가 미래에 어떻게 발전해야 하는지에 대한 초안 제안으로서 P4를 제안한다. 우리는 세 가지 목표를 가지고 있다: (1) 현장에서의 재구성 가능성: 프로그래머가 스위치가 배치된 후에도 패킷 처리 방식을 변경할 수 있어야 한다. (2) 프로토콜 독립성: 스위치가 특정 네트워크 프로토콜에 종속되어서는 안 된다. (3) 타겟 독립성: 프로그래머가 기저 하드웨어의 구체적인 사항과 독립적으로 패킷 처리 기능을 기술할 수 있어야 한다. 예시로서, P4를 사용하여 새로운 계층적 레이블을 추가하도록 스위치를 구성하는 방법을 설명한다.

---

1. 서론

소프트웨어 정의 네트워킹(SDN)은 운영자에게 네트워크에 대한 프로그래밍 방식의 제어 권한을 부여한다. SDN에서 제어 평면은 포워딩 평면과 물리적으로 분리되어 있으며, 하나의 제어 평면이 여러 포워딩 장치를 제어한다. 포워딩 장치는 여러 방식으로 프로그래밍될 수 있지만, OpenFlow와 같은 공통적이고 개방적이며 벤더에 구애받지 않는 인터페이스를 사용하면 제어 평면이 서로 다른 하드웨어 및 소프트웨어 벤더의 포워딩 장치를 제어할 수 있다.

버전	날짜	헤더 필드
OF 1.0	2009년 12월	12개 필드 (이더넷, TCP/IPv4)

버전	날짜	헤더 필드
OF 1.1	2011년 2월	15개 필드 (MPLS, 테이블 간 메타데이터)
OF 1.2	2011년 12월	36개 필드 (ARP, ICMP, IPv6 등)
OF 1.3	2012년 6월	40개 필드
OF 1.4	2013년 10월	41개 필드

표 1: OpenFlow 표준이 인식하는 필드

OpenFlow 인터페이스는 단순하게 시작하여, 수십 개의 헤더 필드(예: MAC 주소, IP 주소, 프로토콜, TCP/UDP 포트 번호 등)에 대해 패킷을 매칭할 수 있는 단일 규칙 테이블의 추상화를 제공하였다. 지난 5년간 명세는 점점 복잡해져, 더 많은 헤더 필드와 다단계 규칙 테이블을 추가하여 스위치가 컨트롤러에 더 많은 기능을 노출할 수 있게 되었다.

새로운 헤더 필드의 확산은 멈출 기미를 보이지 않고 있다. 예를 들어, 데이터센터 네트워크 운영자들은 새로운 형태의 패킷 캡슐화(예: NVGRE, VXLAN, STT)를 점점 더 적용하고자 하며, 이를 위해 새로운 기능을 더 쉽게 확장할 수 있는 소프트웨어 스위치를 배포하는 방식에 의존하고 있다. OpenFlow 명세를 반복적으로 확장하는 대신, 우리는 미래의 스위치가 패킷 파싱 및 헤더 필드 매칭을 위한 유연한 메커니즘을 지원하고, 컨트롤러 애플리케이션이 공통적이고 개방적인 인터페이스(즉, 새로운 "OpenFlow 2.0" API)를 통해 이러한 기능을 활용할 수 있도록 해야 한다고 주장한다. 이러한 일반적이고 확장 가능한 접근 방식은 오늘날의 OpenFlow 1.x 표준보다 더 단순하고, 더 우아하며, 더 미래 지향적일 것이다.

최근의 칩 설계는 이러한 유연성이 테라비트 속도의 맞춤형 ASIC에서 달성될 수 있음을 보여준다 [1, 2, 3]. 이 새로운 세대의 스위치 칩을 프로그래밍하는 것은 결코 쉽지 않다. 각 칩은 마이크로코드 프로그래밍에 가까운 자체적인 저수준 인터페이스를 가지고 있다. 본 논문에서는 프로토콜 독립적 패킷 프로세서(P4)를 프로그래밍하기 위한 고수준 언어의 설계를 개략적으로 제시한다. 그림 1은 P4와 스위치 구성에 사용되는 P4 프로그램, 그리고 고정 기능 스위치의 포워딩 테이블을 채우도록 설계된 기존 API(예: OpenFlow) 간의 관계를 보여준다. P4는 네트워크 프로그래밍의 추상화 수준을 높이며, 컨트롤러와 스위치 간의 일반적인 인터페이스 역할을 할 수 있다. 즉, 우리는 미래 세대의 OpenFlow가 고정된 스위치 설계에 의해 제약받기보다는 컨트롤러가 스위치에게 어떻게 동작할지를 지시할 수 있어야 한다고 믿는다. 핵심 과제는 다양한 하드웨어 및 소프트웨어 스위치에서 표현력의 필요성과 구현의 용이성 사이의 균형을 맞추는 "최적점"을 찾는 것이다. P4를 설계하면서, 우리는 세 가지 주요 목표를 가지고 있다:

- **재구성 가능성.** 컨트롤러는 현장에서 패킷 파싱 및 처리를 재정의할 수 있어야 한다.

- **프로토콜 독립성.** 스위치는 특정 패킷 포맷에 종속되어서는 안 된다. 대신, 컨트롤러는 (i) 특정 이름과 타입의 헤더 필드를 추출하기 위한 패킷 파서와 (ii) 이러한 헤더를 처리하는 타입이 지정된 매치+액션 테이블의 집합을 지정할 수 있어야 한다.
- **타겟 독립성.** C 프로그래머가 기저 CPU의 구체적인 사항을 알 필요가 없는 것처럼, 컨트롤러 프로그래머도 기저 스위치의 세부 사항을 알 필요가 없어야 한다. 대신, 컴파일러가 타겟 독립적인 설명(P4로 작성)을 타겟 종속적인 프로그램(스위치 구성에 사용)으로 변환할 때 스위치의 기능을 고려해야 한다.

본 논문의 구성은 다음과 같다. 먼저 추상적인 스위치 포워딩 모델을 소개한다. 다음으로, 프로토콜 독립적 패킷 처리를 기술하기 위한 새로운 언어의 필요성을 설명한다. 그런 다음, 네트워크 운영자가 새로운 패킷 헤더 필드를 지원하고 다단계로 패킷을 처리하려는 간단한 동기 부여 예제를 제시한다. 이를 사용하여 P4 프로그램이 헤더, 패킷 파서, 다중 매치+액션 테이블, 그리고 이 테이블들 사이의 제어 흐름을 어떻게 지정하는지 살펴본다. 마지막으로, 컴파일러가 P4 프로그램을 타겟 스위치에 어떻게 매핑할 수 있는지 논의한다.

관련 연구. 2011년, Yadav 등 [4]은 OpenFlow를 위한 추상적인 포워딩 모델을 제안했으나 컴파일러에 대한 강조가 적었다. Kangaroo [1]는 프로그래밍 가능한 파싱의 개념을 도입했다. 최근 Song [5]은 프로토콜 망각형 포워딩을 제안했는데, 이는 프로토콜 독립성이라는 우리의 목표를 공유하지만 네트워크 프로세서를 더 많이 겨냥하고 있다. ONF는 스위치의 매칭 기능을 표현하기 위한 테이블 타이핑 패턴을 도입했다 [6]. NOSIX [7]에 대한 최근 연구는 유연한 매치+액션 테이블 명세라는 우리의 목표를 공유하지만, 프로토콜 독립성을 고려하지 않거나 파서, 테이블 및 제어 흐름을 지정하기 위한 언어를 제안하지 않았다. 다른 최근 연구는 모니터링, 혼잡 제어 및 큐 관리를 위한 데이터 평면에 대한 프로그래밍 가능한 인터페이스를 제안한다 [8, 9]. Click 모듈러 라우터 [10]는 소프트웨어에서 유연한 패킷 처리를 지원하지만, 다양한 타겟 하드웨어 스위치에 프로그램을 매핑하지는 않는다.

---

2. 추상 포워딩 모델

우리의 추상 모델(그림 2)에서, 스위치는 프로그래밍 가능한 파서를 통해 패킷을 포워딩한 후, 직렬, 병렬, 또는 양쪽의 조합으로 배열된 다단계의 매치+액션을 거친다. OpenFlow에서 파생된 이 모델은 세 가지 일반화를 제공한다. 첫째, OpenFlow는 고정된 파서를 가정하지만, 우리 모델은 새로운 헤더를 정의할 수 있도록 프로그래밍 가능한 파서를 지원한다. 둘째, OpenFlow는 매치+액션 단계가 직렬이라고 가정하지만, 우리 모델에서는 병렬이거나 직렬일 수 있다. 셋째, 우리 모델은 액션이 스위치가 지원하는 프로토콜 독립적 프리미티브로 구성된다고 가정한다.

우리의 추상 모델은 서로 다른 포워딩 장치(예: 이더넷 스위치, 로드 밸런서, 라우터)와 서로 다른 기술(예: 고정 기능 스위치 ASIC, NPU, 재구성 가능한 스위치, 소프트웨어 스위치, FPGA)에서 패킷이 어떻게 처리되는지를 일반화한다. 이를 통해 공통 언어(P4)를 고안하여 공통 추상 모델의 관점에서

패킷 처리 방식을 표현할 수 있다. 따라서 프로그래머는 타겟 독립적인 프로그램을 작성할 수 있으며, 컴파일러는 비교적 느린 소프트웨어 스위치부터 가장 빠른 ASIC 기반 스위치까지 다양한 포워딩 장치에 매핑할 수 있다.

포워딩 모델은 구성(Configure)과 채우기(Populate)라는 두 가지 유형의 작업으로 제어된다. 구성 작업은 파서를 프로그래밍하고, 매치+액션 단계의 순서를 설정하며, 각 단계에서 처리할 헤더 필드를 지정한다. 구성은 어떤 프로토콜이 지원되고 스위치가 패킷을 어떻게 처리할 수 있는지를 결정한다. 채우기 작업은 구성 중에 지정된 매치+액션 테이블에 항목을 추가(및 제거)한다. 채우기는 특정 시점에 패킷에 적용되는 정책을 결정한다.

본 논문에서는 구성과 채우기가 두 개의 별개 단계라고 가정한다. 특히, 구성 중에는 스위치가 패킷을 처리할 필요가 없다. 그러나 구현체들은 부분적 또는 전체 재구성 중에도 패킷 처리를 허용하여 다운타임 없는 업그레이드를 가능하게 할 것으로 예상한다. 우리 모델은 포워딩을 중단하지 않는 재구성을 의도적으로 허용하고 장려한다.

분명히, 구성 단계는 고정 기능 ASIC 스위치에서는 큰 의미가 없다. 이러한 유형의 스위치에서 컴파일러의 역할은 단순히 칩이 P4 프로그램을 지원할 수 있는지 확인하는 것이다. 대신, 우리의 목표는 [2, 3]에서 설명된 바와 같이 빠른 재구성 가능한 패킷 처리 파이프라인을 향한 일반적인 추세를 포착하는 것이다.

도착하는 패킷은 먼저 파서에 의해 처리된다. 패킷 본문은 별도로 버퍼링되며 매칭에는 사용할 수 없다고 가정한다. 파서는 헤더에서 필드를 인식하고 추출하며, 이를 통해 스위치가 지원하는 프로토콜을 정의한다. 이 모델은 프로토콜 헤더의 의미에 대해 어떠한 가정도 하지 않으며, 파싱된 표현이 매칭 및 액션이 작동하는 필드의 집합을 정의한다는 점만 중요하다.

추출된 헤더 필드는 매치+액션 테이블로 전달된다. 매치+액션 테이블은 인그레스(수신)와 이그레스(송신)로 나뉜다. 둘 다 패킷 헤더를 수정할 수 있지만, 인그레스 매치+액션은 이그레스 포트를 결정하고 패킷이 놓일 큐를 결정한다. 인그레스 처리에 따라 패킷은 포워딩되거나, 복제되거나(멀티캐스트, 스팬, 또는 제어 평면으로), 드롭되거나, 흐름 제어를 트리거할 수 있다. 이그레스 매치+액션은 패킷 헤더에 인스턴스별 수정을 수행한다 - 예를 들어 멀티캐스트 복사본의 경우. 액션 테이블(카운터, 폴리서 등)은 플로우와 연관되어 프레임 간 상태를 추적할 수 있다.

패킷은 메타데이터라고 불리는 추가 정보를 단계 간에 전달할 수 있으며, 이는 패킷 헤더 필드와 동일하게 취급된다. 메타데이터의 예로는 인그레스 포트, 전송 대상 및 큐, 패킷 스케줄링에 사용할 수 있는 타임스탬프, 그리고 가상 네트워크 식별자와 같이 패킷의 파싱된 표현을 변경하지 않고 테이블 간에 전달되는 데이터가 있다.

큐잉 규율은 현재 OpenFlow와 동일한 방식으로 처리된다: 액션이 패킷을 큐에 매핑하며, 큐는 특정 서비스 규율을 받도록 구성된다. 서비스 규율(예: 최소 속도, DRR)은 스위치 구성의 일부로 선택된다.

본 논문의 범위를 벗어나지만, 프로그래머가 새로운 또는 기존의 혼잡 제어 프로토콜을 구현할 수 있도록 액션 프리미티브를 추가할 수 있다. 예를 들어, 새로운 조건에 따라 ECN 비트를 설정하도록 스

위치를 프로그래밍하거나, 매치+액션 테이블을 사용하여 독자적인 혼잡 제어 메커니즘을 구현할 수 있다.

---

3. 프로그래밍 언어

우리는 추상 포워딩 모델을 사용하여 스위치의 구성 방식과 패킷 처리 방식을 표현하는 언어를 정의한다. 이 논문의 주요 목표는 P4 프로그래밍 언어를 제안하는 것이다. 그러나 많은 언어가 가능하며, 이들은 우리가 여기서 설명하는 공통 특성을 공유할 것이다. 예를 들어, 언어는 파서가 어떤 패킷 포맷을 예상해야 하는지 알 수 있도록 파서가 프로그래밍되는 방식을 표현할 필요가 있다. 따라서 프로그래머는 어떤 헤더 타입이 가능한지 선언할 수 있어야 한다. 예를 들어, 프로그래머는 IPv4 헤더의 포맷과 IP 헤더 다음에 올 수 있는 합법적인 헤더를 지정할 수 있다. 이는 P4에서 합법적인 헤더 타입을 선언하여 파싱을 정의하는 것을 동기 부여한다. 마찬가지로, 프로그래머는 패킷 헤더가 어떻게 처리되어야 하는지 표현할 필요가 있다. 예를 들어, TTL 필드는 감소되어야 하고, 새로운 터널 헤더가 추가되어야 할 수 있으며, 체크섬이 계산되어야 할 수 있다. 이는 선언된 헤더 타입과 프리미티브 액션 집합을 사용하여 헤더 필드 처리를 기술하기 위한 명령형 제어 흐름 프로그램의 사용을 동기 부여한다.

Click [10]과 같은 언어를 사용할 수도 있는데, 이는 임의의 C++ 모듈로 구성된 스위치를 구축한다. Click은 매우 표현력이 풍부하며 CPU 커널에서 패킷이 처리되는 방식을 표현하는 데 매우 적합하다. 그러나 우리의 필요에는 충분히 제약되지 않는다 - 전용 하드웨어에서 파스-매치-액션 파이프라인을 미러링하는 언어가 필요하다. 또한 Click은 컨트롤러-스위치 아키텍처를 위해 설계되지 않았으며, 따라서 잘 타이핑된 규칙으로 동적으로 채워지는 매치+액션 테이블을 프로그래머가 기술할 수 없다. 마지막으로 Click은 병렬 실행을 제약하는 종속성을 추론하기 어렵게 만든다 - 이에 대해 다음에서 논의한다.

패킷 처리 언어는 프로그래머가 헤더 필드 간의 직렬 종속성을 (명시적 또는 암묵적으로) 표현할 수 있어야 한다. 종속성은 어떤 테이블이 병렬로 실행될 수 있는지를 결정한다. 예를 들어, IP 라우팅 테이블과 ARP 테이블은 데이터 종속성이 있기 때문에 순차 실행이 필요하다. 종속성은 테이블 종속성 그래프(TDG)를 분석하여 식별할 수 있다. 이 그래프는 테이블 간의 필드 입력, 액션 및 제어 흐름을 기술한다. 그림 3은 L2/L3 스위치에 대한 테이블 종속성 그래프의 예를 보여준다. TDG 노드는 매치+액션 테이블에 직접 매핑되며, 종속성 분석은 각 테이블이 파이프라인의 어디에 위치해야 하는지를 식별한다. 안타깝게도 TDG는 대부분의 프로그래머에게 쉽게 접근 가능하지 않다. 프로그래머는 그래프보다는 명령형 구조를 사용하여 패킷 처리 알고리즘을 생각하는 경향이 있다.

이로 인해 2단계 컴파일 과정을 제안하게 된다. 최상위 수준에서 프로그래머는 제어 흐름을 나타내는 명령형 언어(P4)를 사용하여 패킷 처리 프로그램을 표현한다. 그 아래에서 컴파일러는 P4 표현을 TDG로 변환하여 종속성 분석을 용이하게 하고, 그런 다음 TDG를 특정 스위치 타겟에 매핑한다. P4

는 P4 프로그램을 TDG로 쉽게 변환할 수 있도록 설계되었다. 요약하면, P4는 종속성을 추론하고 하드웨어에 매핑하기 어렵게 만드는 Click의 일반성과, 프로토콜 처리를 재구성하는 것이 불가능한 OpenFlow 1.0의 비유연성 사이의 "최적점"이라고 할 수 있다.

---

4. P4 언어 예제

우리는 간단한 예제를 심층적으로 살펴봄으로써 P4를 탐구한다. 많은 네트워크 배치에서 엣지와 코어를 구분한다. 엔드호스트는 엣지 장치에 직접 연결되고, 엣지 장치는 고대역폭 코어에 의해 상호 연결된다. MPLS [11]와 PortLand [12]와 같은 전체 프로토콜이 이 아키텍처를 지원하도록 설계되었으며, 주로 코어에서의 포워딩을 단순화하는 것을 목적으로 한다.

2계층 코어에 연결된 엣지의 탑오브랙(ToR) 스위치를 사용한 L2 네트워크 배포 예를 생각해보자. 엔드호스트의 수가 증가하고 코어 L2 테이블이 오버플로되고 있다고 가정한다. MPLS는 코어를 단순화하는 옵션이지만, 다중 태그를 사용한 레이블 분배 프로토콜을 구현하는 것은 어려운 일이다. PortLand는 흥미롭지만 MAC 주소를 재작성해야 하므로 기존 네트워크 디버깅 도구가 작동하지 않을 수 있으며 ARP 요청에 응답할 새로운 에이전트가 필요하다.

P4를 사용하면 네트워크 아키텍처에 최소한의 변경으로 맞춤형 솔루션을 표현할 수 있다. 우리의 장난감 예제를 mTag라고 부른다: 이것은 PortLand의 계층적 라우팅과 간단한 MPLS 유사 태그를 결합한다. 코어를 통한 경로는 4개의 1바이트 필드로 구성된 32비트 태그로 인코딩된다. 32비트 태그는 "소스 경로" 또는 목적지 로케이터(PortLand의 Pseudo MAC과 같은)를 전달할 수 있다. 각 코어 스위치는 태그의 한 바이트만 검사하고 그 정보에 따라 스위칭하면 된다. 우리의 예에서 태그는 첫 번째 ToR 스위치에 의해 추가되지만, 엔드호스트 NIC에 의해 추가될 수도 있다.

mTag 예제는 P4 언어에 집중하기 위해 의도적으로 매우 단순하다. 실제 전체 스위치를 위한 P4 프로그램은 훨씬 더 복잡할 것이다.

4.1 P4 개념

P4 프로그램은 다음과 같은 핵심 구성 요소의 정의를 포함한다:

- **헤더:** 헤더 정의는 일련의 필드의 시퀀스와 구조를 기술한다. 필드 너비와 필드 값에 대한 제약의 명세를 포함한다.
- **파서:** 파서 정의는 패킷 내의 헤더와 유효한 헤더 시퀀스를 식별하는 방법을 지정한다.
- **테이블:** 매치+액션 테이블은 패킷 처리를 수행하는 메커니즘이다. P4 프로그램은 테이블이 매칭할 수 있는 필드와 실행할 수 있는 액션을 정의한다.

- **액션:** P4는 더 단순한 프로토콜 독립적 프리미티브로부터 복잡한 액션의 구성을 지원한다. 이러한 복잡한 액션은 매치+액션 테이블 내에서 사용할 수 있다.
- **제어 프로그램:** 제어 프로그램은 패킷에 적용되는 매치+액션 테이블의 순서를 결정한다. 간단한 명령형 프로그램이 매치+액션 테이블 간의 제어 흐름을 기술한다.

다음으로, 이러한 각 구성 요소가 P4에서 이상화된 mTag 프로세서의 정의에 어떻게 기여하는지 보여준다.

4.2 헤더 포맷

설계는 헤더 포맷의 명세로 시작한다. 이를 위해 여러 도메인 특화 언어가 제안되었다 [13, 14, 15]. P4는 이들로부터 많은 아이디어를 빌려온다. 일반적으로, 각 헤더는 필드 이름과 너비의 정렬된 목록을 선언하여 지정된다. 선택적 필드 주석은 값 범위 또는 가변 길이 필드의 최대 길이에 대한 제약을 허용한다. 예를 들어, 표준 이더넷 및 VLAN 헤더는 다음과 같이 지정된다:

```
header ethernet {
    fields {
        dst_addr : 48; // 비트 단위 너비
        src_addr : 48;
        ethertype : 16;
    }
}

header vlan {
    fields {
        pcp : 3;
        cfi : 1;
        vid : 12;
        ethertype : 16;
    }
}
```

mTag 헤더는 기존 선언을 변경하지 않고 추가할 수 있다. 필드 이름은 코어에 두 계층의 집계가 있음을 나타낸다. 각 코어 스위치는 계층 구조에서의 위치와 이동 방향(상향 또는 하향)에 따라 이 바이트 중 하나를 검사하는 규칙으로 프로그래밍된다.

```
header mTag {
    fields {
        up1 : 8;
        up2 : 8;
        down1 : 8;
        down2 : 8;
        ethertype : 16;
```

```
    }
}
```

4.3 패킷 파서

P4는 기저 스위치가 패킷 헤더를 처음부터 끝까지 순회하면서 필드 값을 추출하는 상태 머신을 구현할 수 있다고 가정한다. 추출된 필드 값은 처리를 위해 매치+액션 테이블로 전송된다.

P4는 이 상태 머신을 하나의 헤더에서 다음 헤더로의 전이 집합으로 직접 기술한다. 각 전이는 현재 헤더의 값에 의해 트리거될 수 있다. 예를 들어, mTag 상태 머신을 다음과 같이 기술한다:

```
parser start {
    ethernet;
}

parser ethernet {
    switch(ethertype) {
        case 0x8100: vlan;
        case 0x9100: vlan;
        case 0x800: ipv4;
        // 기타 경우
    }
}

parser vlan {
    switch(ethertype) {
        case 0xaaaa: mTag;
        case 0x800: ipv4;
        // 기타 경우
    }
}

parser mTag {
    switch(ethertype) {
        case 0x800: ipv4;
        // 기타 경우
    }
}
```

파싱은 `start` 상태에서 시작하여 명시적인 `stop` 상태에 도달하거나 처리되지 않은 경우(오류로 표시될 수 있음)를 만날 때까지 진행된다. 새로운 헤더에 대한 상태에 도달하면, 상태 머신은 해당 헤더의 명세를 사용하여 헤더를 추출하고 다음 전이를 식별한다. 추출된 헤더는 스위치 파이프라인의 후반부에서 매치+액션 처리를 위해 전달된다.

mTag의 파서는 매우 단순하다: 4개의 상태만 있다. 실제 네트워크의 파서는 훨씬 더 많은 상태를 필요로 한다. 예를 들어 Gibb 등이 정의한 파서 [16, 그림 3(e)]는 100개 이상의 상태로 확장된다.

4.4 테이블 명세

다음으로, 프로그래머는 정의된 헤더 필드가 매치+액션 단계에서 어떻게 매칭되어야 하는지(예: 정확 매칭, 범위, 와일드카드?) 및 매칭이 발생했을 때 어떤 액션을 수행해야 하는지를 기술한다.

간단한 mTag 예에서, 엣지 스위치는 L2 목적지와 VLAN ID를 매칭하고 헤더에 추가할 mTag를 선택한다. 프로그래머는 이 필드에 매칭하고 mTag 헤더를 추가하는 액션을 적용하기 위한 테이블을 정의한다(아래 참조). **reads** 속성은 매칭할 필드와 매칭 타입(정확, 삼진 등)을 선언한다. **actions** 속성은 테이블이 패킷에 적용할 수 있는 가능한 액션을 나열한다. **max_size** 속성은 테이블이 지원해야 하는 항목 수를 지정한다.

테이블 명세는 컴파일러가 필요한 메모리 양과 테이블을 구현할 메모리 타입(예: TCAM 또는 SRAM)을 결정할 수 있게 한다.

```
table mTag_table {
    reads {
        ethernet.dst_addr : exact;
        vlan.vid : exact;
    }
    actions {
        // 실행 시 항목은 mTag 액션의
        // 매개변수로 프로그래밍됨. 아래 참조.
        add_mTag;
    }
    max_size : 20000;
}
```

완전성과 후속 논의를 위해, 제어 프로그램(§4.6)이 참조하는 다른 테이블의 간략한 정의를 제시한다.

```
table source_check {
    // 코어에 연결된 포트에서만 mTag 검증
    reads {
        mtag : valid; // mTag가 파싱되었는가?
        metadata.ingress_port : exact;
    }
    actions { // 각 테이블 항목은 *하나의* 액션을 지정
        // 부적절한 mTag이면 CPU로 전송
        fault_to_cpu;

        // mTag가 발견되면 제거하고 메타데이터에 기록
        strip_mtag;

        // 그렇지 않으면 패킷이 계속 진행
        pass;
    }
```

```
    max_size : 64; // 포트당 하나의 규칙
}

table local_switching {
    // 목적지를 읽고 로컬인지 확인
    // 미스가 발생하면 mTag 테이블로 이동
}

table egress_check {
    // 이그레스가 확인되었는지 검증
    // 태그가 있는 패킷을 다시 태깅하지 않음
    // 이그레스를 읽고 패킷이 mTag 처리되었는지 확인
}
```

4.5 액션 명세

P4는 더 복잡한 액션이 구축되는 프리미티브 액션의 집합을 정의한다. 각 P4 프로그램은 액션 프리미티브로 구성된 액션 함수의 집합을 선언한다. 이러한 액션 함수는 테이블 명세와 채우기를 단순화한다. P4는 액션 함수 내에서 프리미티브의 병렬 실행을 가정한다. (병렬 실행이 불가능한 스위치는 시맨틱스를 에뮬레이션할 수 있다.)

위에서 언급한 `add_mTag` 액션은 다음과 같이 구현된다:

```
action add_mTag(up1, up2, down1, down2, egr_spec) {
    add_header(mTag);
    // VLAN의 ethertype을 mTag로 복사
    copy_field(mTag.ethertype, vlan.ethertype);
    // mTag를 알리기 위해 VLAN의 ethertype 설정
    set_field(vlan.ethertype, 0xaaaa);
    set_field(mTag.up1, up1);
    set_field(mTag.up2, up2);
    set_field(mTag.down1, down1);
    set_field(mTag.down2, down2);

    // 목적지 이그레스 포트도 설정
    set_field(metadata.egress_spec, egr_spec);
}
```

액션이 매개변수를 필요로 하는 경우(예: mTag의 up1 값), 이는 실행 시에 매치 테이블에서 제공된다.

이 예에서, 스위치는 VLAN 태그 뒤에 mTag를 삽입하고, VLAN 태그의 Ethertype을 mTag로 복사하여 뒤에 오는 것을 나타내고, VLAN 태그의 Ethertype을 mTag를 알리기 위해 0xaaaa로 설정한다. 여기에 표시되지 않은 것은 패킷에서 mTag를 제거하는 역방향 액션 명세와 엣지 스위치에서 이 액션을 적용하는 테이블이다.

P4의 프리미티브 액션은 다음을 포함한다: - **set_field:** 헤더의 특정 필드를 값으로 설정. 마스크된 집합도 지원. - **copy_field:** 한 필드를 다른 필드로 복사. - **add_header:** 특정 헤더 인스턴스(및 모든 필드)를 유효하게 설정. - **remove_header:** 패킷에서 헤더(및 모든 필드)를 삭제("팝"). - **increment:** 필드의 값을 증가 또는 감소. - **checksum:** 헤더 필드의 일부 집합에 대해 체크섬을 계산(예: IPv4 체크섬).

대부분의 스위치 구현은 지정된 패킷 포맷과 일치하는 헤더 수정만 허용하도록 액션 처리를 제한할 것으로 예상한다.

4.6 제어 프로그램

테이블과 액션이 정의되면, 남은 유일한 작업은 한 테이블에서 다음 테이블로의 제어 흐름을 지정하는 것이다. 제어 흐름은 함수, 조건문 및 테이블 참조의 집합으로 프로그램으로 지정된다.

그림 4는 엣지 스위치에서의 mTag 구현을 위한 원하는 제어 흐름의 그래픽 표현을 보여준다. 파싱 후, `source_check` 테이블은 수신된 패킷과 인그레스 포트 간의 일관성을 검증한다. 예를 들어, mTag는 코어 스위치에 연결된 포트에서만 볼 수 있어야 한다. `source_check`는 패킷에서 mTag를 제거하고, 패킷에 mTag가 있었는지를 메타데이터에 기록한다. 파이프라인 후반부의 테이블은 이 메타데이터를 매칭하여 패킷의 재태깅을 방지할 수 있다.

그런 다음 `local_switching` 테이블이 실행된다. 이 테이블이 "미스"하면, 패킷이 로컬에 연결된 호스트로 가는 것이 아님을 나타낸다. 이 경우 `mTag_table`(위에서 정의)이 패킷에 적용된다. 로컬 및 코어 포워딩 제어 모두 `egress_check` 테이블에 의해 처리될 수 있으며, 이 테이블은 알 수 없는 목적지의 경우를 처리하고 SDN 제어 스택에 알림을 보낸다.

이 패킷 처리 파이프라인의 명령형 표현은 다음과 같다:

```
control main() {
    // mTag 상태와 포트가 일관성 있는지 검증
    table(source_check);

    // source_check에서 오류가 없으면 계속
    if (!defined(metadata.ingress_error)) {
        // 엔드호스트로의 스위칭 시도
        table(local_switching);

        if (!defined(metadata.egress_spec)) {
            // 알려진 로컬 호스트가 아님; mTag 처리 시도
            table(mTag_table);
        }
    }

    // 알 수 없는 이그레스 상태 또는
    // mTag를 사용한 잘못된 재태깅 확인
```

```
    table(egress_check);
}
```

---

5. P4 프로그램 컴파일

네트워크에서 P4 프로그램을 구현하려면, 타겟 독립적인 설명을 타겟 스위치의 특정 하드웨어 또는 소프트웨어 플랫폼에 매핑하는 컴파일러가 필요하다. 이를 위해 타겟의 리소스를 할당하고 장치에 적합한 구성을 생성해야 한다.

5.1 패킷 파서 컴파일

프로그래밍 가능한 파서를 가진 장치의 경우, 컴파일러는 파서 설명을 파싱 상태 머신으로 변환하고, 고정 파서의 경우에는 파서 설명이 타겟의 파서와 일치하는지만 검증한다. 상태 머신과 상태 테이블 항목 생성에 대한 세부 사항은 [16]에서 찾을 수 있다.

표 2는 파서(§4.3)의 vlan 및 mTag 섹션에 대한 상태 테이블 항목을 보여준다. 각 항목은 현재 상태, 매칭할 필드 값, 다음 상태를 지정한다. 간결함을 위해 다른 열은 생략되었다.

현재 상태	조회 값	다음 상태
vlan	0xaaaa	mTag
vlan	0x800	ipv4
vlan	*	stop
mTag	0x800	ipv4
mTag	*	stop

표 2: mTag 예제의 파서 상태 테이블 항목

5.2 제어 프로그램 컴파일

§4.6의 명령형 제어 흐름 표현은 스위치의 논리적 포워딩 동작을 지정하는 편리한 방법이지만, 테이블 간의 종속성이나 동시성의 기회를 명시적으로 나타내지 않는다. 따라서 컴파일러를 사용하여 제어 프로그램을 분석하고 종속성을 식별하며 헤더 필드를 병렬로 처리할 기회를 찾는다. 마지막으로, 컴파일러는 스위치에 대한 타겟 구성을 생성한다. 많은 잠재적 타겟이 있다: 예를 들어 소프트웨어 스위치 [17], 멀티코어 소프트웨어 스위치 [18], NPU [19], 고정 기능 스위치 [20], 또는 재구성 가능한 매치 테이블(RMT) 파이프라인 [2].

§3에서 논의한 바와 같이, 컴파일러는 2단계 컴파일 과정을 따른다. 먼저 P4 제어 프로그램을 중간 테이블 종속성 그래프 표현으로 변환하여 테이블 간의 종속성을 분석하고 결정한다. 그런 다음 타겟 특화 백엔드가 이 그래프를 스위치의 특정 리소스에 매핑한다.

mTag 예가 어떻게 다른 종류의 스위치에서 구현될 수 있는지를 간략히 살펴본다:

소프트웨어 스위치: 소프트웨어 스위치는 완전한 유연성을 제공한다: 테이블 수, 테이블 구성, 파싱이 모두 소프트웨어 제어 하에 있다. 컴파일러는 mTag 테이블 그래프를 스위치 테이블에 직접 매핑한다. 컴파일러는 각 테이블의 테이블 너비, 높이, 매칭 기준(예: 정확, 프리픽스, 와일드카드)을 제한하기 위해 테이블 타입 정보를 사용한다. 컴파일러는 소프트웨어 데이터 구조로 삼진 또는 프리픽스 매칭을 최적화할 수도 있다.

RAM과 TCAM이 있는 하드웨어 스위치: 컴파일러는 해싱을 구성하여 엣지 스위치에서 `mTag_table`에 대한 효율적인 정확 매칭을 RAM을 사용하여 수행할 수 있다. 반대로, 태그 비트의 부분집합에 매칭하는 코어 mTag 포워딩 테이블은 TCAM에 매핑될 수 있다.

병렬 테이블을 지원하는 스위치: 컴파일러는 데이터 종속성을 감지하고 테이블을 병렬 또는 직렬로 배열할 수 있다. mTag 예에서, `mTag_table`과 `local_switching` 테이블은 mTag 설정 액션의 실행까지 병렬로 실행할 수 있다.

파이프라인 끝에서 액션을 적용하는 스위치: 파이프라인 끝에서만 액션 처리를 수행하는 스위치의 경우, 컴파일러는 최종 쓰기를 수행하는 데 사용되는 중간 메타데이터를 생성하도록 중간 단계에 지시할 수 있다. mTag 예에서, mTag의 추가 또는 제거 여부는 메타데이터로 표현될 수 있다.

테이블이 적은 스위치: 컴파일러는 많은 수의 P4 테이블을 적은 수의 물리적 테이블에 매핑할 수 있다. mTag 예에서, 로컬 스위칭은 mTag 테이블과 결합될 수 있다. 컨트롤러가 실행 시에 새로운 규칙을 설치할 때, 컴파일러의 규칙 변환기는 두 P4 테이블의 규칙을 "합성"하여 단일 물리적 테이블에 대한 규칙을 생성할 수 있다.

---

6. 결론

SDN의 약속은 단일 제어 평면이 전체 스위치 네트워크를 직접 제어할 수 있다는 것이다. OpenFlow는 단일의, 벤더에 구애받지 않는 API를 제공함으로써 이 목표를 지원한다. 그러나 오늘날의 OpenFlow는 미리 결정된 헤더 필드 집합을 인식하고 소수의 미리 정의된 액션을 사용하여 패킷을 처리하는 고정 기능 스위치를 대상으로 한다. 제어 평면은 패킷이 제어 애플리케이션의 요구를 가장 잘 충족하도록 처리되는 방법을 표현할 수 없다.

우리는 기능이 명세되고 현장에서 변경될 수 있는 더 유연한 스위치를 향한 발걸음을 제안한다. 프로그래머가 구현 세부 사항을 걱정하지 않고 포워딩 평면이 패킷을 처리하는 방식을 결정한다. 컴파일

러가 명령형 프로그램을 테이블 종속성 그래프로 변환하며, 이 그래프는 최적화된 하드웨어 구현을 포함하여 많은 특정 타겟 스위치에 매핑될 수 있다.

우리는 이것이 OpenFlow 2.0에 대한 논의에 기여하기 위해 초안 제안으로 설계된 첫 번째 단계에 불과하다는 점을 강조한다. 이 제안에서, 스위치의 여러 측면(예: 혼잡 제어 프리미티브, 큐잉 규율, 트래픽 모니터링)은 아직 정의되지 않았다. 그러나 우리는 구성 언어를 갖고 특정 타겟에 대한 저수준 구성을 생성하는 컴파일러를 갖는 접근 방식이 더 큰 유연성을 제공하는 미래의 스위치로 이끌고, 소프트웨어 정의 네트워크의 잠재력을 발휘할 것이라고 믿는다.

---

7. 참고문헌

[1] C. Kozanitis, J. Huber, S. Singh, G. Varghese, "Leaping multiple headers in a single bound: Wire-speed parsing using the Kangaroo system," IEEE INFOCOM, pp. 830–838, 2010.

[2] P. Bosshart, G. Gibb, H.-S. Kim, G. Varghese, N. McKeown, M. Izzard, F. Mujica, M. Horowitz, "Forwarding metamorphosis: Fast programmable match-action processing in hardware for SDN," ACM SIGCOMM, 2013.

[3] "Intel Ethernet Switch Silicon FM6000."

[4] N. Yadav and D. Cohn, "OpenFlow Primitive Set," July 2011.

[5] H. Song, "Protocol-oblivious forwarding: Unleash the power of SDN through a future-proof forwarding plane," SIGCOMM HotSDN Workshop, Aug. 2013.

[6] "Openflow forwarding abstractions working group charter," Apr. 2013.

[7] M. Raju, A. Wundsam, and M. Yu, "NOSIX: A lightweight portability layer for the SDN OS," ACM SIGCOMM Computer Communications Review, 2014.

[8] V. Jeyakumar, M. Alizadeh, C. Kim, and D. Mazieres, "Tiny packet programs for low-latency network control and monitoring," ACM SIGCOMM HotNets Workshop, Nov. 2013.

[9] A. Sivaraman, K. Winstein, S. Subramanian, and H. Balakrishnan, "No silver bullet: Extending SDN to the data plane," ACM SIGCOMM HotNets Workshop, Nov. 2013.

[10] E. Kohler, R. Morris, B. Chen, J. Jannotti, and M. F. Kaashoek, "The Click modular router," ACM Transactions on Computer Systems, vol. 18, pp. 263–297, Aug. 2000.

[11] "Multiprotocol Label Switching Charter."

[12] R. Niranjan Mysore, A. Pamboris, N. Farrington, N. Huang, P. Miri, S. Radhakrishnan, V. Subramanya, and A. Vahdat, "PortLand: A scalable fault-tolerant layer 2 data center network fabric," ACM SIGCOMM, pp. 39–50, Aug. 2009.

[13] P. McCann and S. Chandra, "PacketTypes: Abstract specifications of network protocol messages," ACM SIGCOMM, pp. 321–333, Aug. 2000.

[14] G. Back, "DataScript - A specification and scripting language for binary data," Generative Programming and Component Engineering, vol. 2487, pp. 66–77, Lecture Notes in Computer Science, 2002.

[15] K. Fisher and R. Gruber, "PADS: A domain specific language for processing ad hoc data," ACM Conference on Programming Language Design and Implementation, pp. 295–304, June 2005.

[16] G. Gibb, G. Varghese, M. Horowitz, and N. McKeown, "Design principles for packet parsers," ANCS, pp. 13–24, 2013.

[17] "Open vSwitch website."

[18] D. Zhou, B. Fan, H. Lim, M. Kaminsky, and D. G. Andersen, "Scalable, high performance ethernet forwarding with CuckooSwitch," CoNext, pp. 97–108, 2013.

[19] "EZChip 240-Gigabit Network Processor for Carrier Ethernet Applications."

[20] "Broadcom BCM56850 Series."